ACAS: Adaptive Control of Inference-Time Activation Sparsity for On-Device LLMs

Ethan Lin, *Student Member, IEEE,* Youngmin Yi, *Member, IEEE,* and Hoeseok Yang, *Senior Member, IEEE*

***Abstract*—The growing demand for on-device large language model (LLM) inference—driven by concerns over privacy, security, and availability—calls for improved computational and memory efficiency. Activation sparsity, arising from the properties of activation functions, offers a promising path but typically relies on statically tuned hyperparameters to balance accuracy and resource savings. We introduce ACAS (Adaptive Control of Activation Sparsity), a lightweight inference-time control mechanism that dynamically adjusts these sparsity parameters without offline retraining or manual calibration. ACAS exploits underutilized CPU resources in conventional CPU–GPU systems, using an online controller instantiated with either a Proportional–Integral–Derivative (PID) controller or a lightweight reinforcement-learning (RL) agent, to periodically assess the quality of sparsity decisions and update parameters in real time. We further extend ACAS with a flexible update mechanism that adapts the update frequency based on runtime conditions. Applied to state-of-the-art output activation sparsity frameworks, ACAS consistently achieves stronger accuracy–latency trade-offs than statically tuned baselines in both ReLU-fied and SiLU-based LLMs, with negligible power and energy overhead and no observable power spikes, and offers new insights into activation sparsity behavior in on-device settings, including context-dependent optimal configurations and layer-specific sensitivity. It enables a practical design space exploration tool for sparsity configurations, and its stability is verified through analyses of convergence behavior under varying initializations and gain perturbations.**

***Index Terms*—On-device LLM inference, activation sparsity, PID control, dynamic parameter tuning, latency–accuracy trade-off**

I. Introduction

Since the introduction of the Transformer architecture [1], large language models (LLMs) have rapidly advanced and are now crucial for many real-world applications, making efficient on-device deployment increasingly important [2]. To meet this demand, researchers have pursued various optimization techniques for general-purpose LLMs, including quantization [3], [4], pruning [5], [6], knowledge distillation (KD) [7], [8], and other compression methods. During LLM inference, the *decoding* stage, rather than the parallel *prefill*, becomes the main bottleneck. In on-device settings with batch size of one, the Multi-Layer Perceptron (MLP) block often dominates, as the weight loads for General Matrix–Vector multiplication (GEMV) operations can outweigh the latency of multi-head attention (MHA) [9], [10].

E. Lin and H. Yang are with the Department of Electrical and Computer Engineering, Santa Clara University, Santa Clara, CA 95053 USA.

Y. Yi is with the Department of Artificial Intelligence, Sogang University, Seoul 04107, Republic of Korea.

Youngmin Yi and Hoeseok Yang are the corresponding authors (e-mail: ymyi@sogang.ac.kr; hoeseok.yang@scu.edu).

Recent work explores activation sparsity to optimize MLP blocks [9]–[18], offering a complementary approach that can be combined with other optimizations. Two variants are commonly studied: *input* activation sparsity [9], [17], where elements of the hidden state are zeroed so that the corresponding weight columns need not contribute to the product, and *output* activation sparsity [10]–[16], [18], [19], where many post-activation values vanish, allowing the associated weight rows in the weight matrix to be skipped if those zero values can be predicted in advance. While output activation sparsity directly enables row-skipping, input activation sparsity faces an important limitation: unless weights are stored in column-major format or explicitly restructured, the values to be skipped correspond to entire columns of a weight matrix, which are not stored contiguously in memory on typical GPU or CPU architectures. As a result, memory load skipping becomes difficult, and the benefit of activation sparsity is largely restricted to skipping computations.

Since the introduction of *ReLU-fication* [11], which deliberately replaces smoother activations (e.g., SiLU or GELU) with ReLU to maximize the post-activation zeros, interest in output activation sparsity has grown even more rapidly. A notable observation is that most, if not all, of the recent studies on activation sparsity in LLMs involve *tunable* hyperparameters related to sparsity prediction. These parameters typically serve to control how aggressively activation sparsity is exploited, effectively acting as a key knob for exploring the trade-off between inference quality and resource efficiency in LLMs. However, the resulting search space is extremely large. For instance, selecting a single sparsity parameter per layer with even 100 candidate values already yields a space of size 100^N for a model with N layers, about 10^{64} for the 32 layers of LLaMA-2-7B, making exhaustive or repeated tuning computationally prohibitive and not scalable across models or tasks. In practice, they are often determined via meta-heuristic search [17], through test or calibration runs [14], [18], manually configured by designers [10], or learned via training [12], [13]. However, such reliance on manually or empirically tuned parameters presents a significant limitation: when the underlying LLM changes or the downstream task is altered, the tuning process typically needs to be repeated, which can be not only costly and time-consuming but also detrimental to portability or general applicability.

In this work, we introduce *ACAS* (Adaptive Control of Activation Sparsity), which autonomously and adaptively controls sparsity-related parameters at inference time. In typical heterogeneous computing environments, where the inference workload is primarily offloaded to GPUs, host processors

often remain underutilized or idle. Our approach exploits these otherwise idle resources by running a lightweight, Proportional-Integral-Derivative-(PID)-controller-based online optimizer that monitors the GPU inference status and dynamically adjusts the sparsity-related parameters. While we adopt PID control in this work due to its simplicity, low overhead, and widespread use in runtime optimization, the proposed ACAS framework is not tied to any specific optimization algorithm. As demonstrated in Section V-C, it can also be integrated with other strategies, such as reinforcement learning (RL)-based controllers.

In many accelerator-based LLM inference systems, the host CPU is primarily responsible for orchestration, kernel launches, and runtime management, while the dominant computation is executed on the accelerator. ACAS leverages this commonly available execution headroom to perform lightweight control computations concurrently with inference. The proposed framework therefore targets the common deployment scenario in which limited host CPU resources remain available during GPU-based inference.

To demonstrate the generality of our proposed method, we apply it to both ReLU-fied [10], [18] and non-ReLU-based [14] output-activation-sparsity techniques. Across multiple standard LLM benchmark suites, applying ACAS enables these techniques to explore the design space defined by the trade-off between inference speed and output quality more effectively than when their parameters are fixed through manual tuning or offline calibration runs, and in some cases even to outperform such statically tuned settings in both accuracy and latency.

Building on the ability of ACAS to optimize activation-sparsity parameters dynamically during inference, we further extend an existing approach for non-ReLU-fied LLMs [14]. The original method relies on a simple thresholding strategy in which activations with small magnitudes are forced to zero. We augment this strategy with a predictive skip mechanism like [10], [18]: when the input to an activation can be confidently estimated to be a large-magnitude negative value, the computation for that unit can be safely skipped. This additional prediction improves the overall efficiency of skipping and broadens the applicability of activation-sparsity techniques to models that do not employ ReLU activations.

The contributions of this work can be summarized as follows:

- We propose ACAS, a lightweight runtime adaptation framework that autonomously adjusts sparsity-related parameters at inference time in CPU–GPU heterogeneous systems. Although instantiated with a PID controller in this work, the framework is compatible with other adaptive optimization strategies, including RL.
- We apply ACAS to three recent activation sparsity techniques and demonstrate that it more effectively explores the trade-off between inference speed and output quality than static approaches.
- We propose an enhanced method to exploit activation sparsity in non-ReLU–based LLMs, leveraging ACAS's online adaptation capability.

Beyond demonstrating performance improvements, we conduct a comprehensive evaluation of ACAS from a systems perspective. Specifically, we examine (i) the necessity of runtime adaptation by analyzing how optimal sparsity configurations vary across prompts and workloads/benchmarks, (ii) the overhead of the control loop in terms of latency, resource usage, and energy consumption, and (iii) the stability of the feedback mechanism, including its impact on transient behaviors such as latency spikes and power fluctuations during adaptation. We further explore extensions to dynamically adjust the feedback period for improved efficiency, and investigate the potential of ACAS as a lightweight design space exploration (DSE) tool for navigating accuracy–efficiency trade-offs.

The remainder of this paper is organized as follows. We first review activation sparsity and recent work that exploits it for efficient on-device LLM inference in Section II. Then, in Section III, we present the proposed PID-based ACAS framework for adaptive control of activation sparsity, followed by a discussion of how existing techniques can be extended by applying ACAS (Section IV). In Section V, we validate the effectiveness of the proposed technique using standard LLM benchmark suites. Finally, we conclude with a brief summary and discussion.

II. Background and Related Work

MLP Block: While the proposed technique is not limited to any specific MLP block architecture, we present it using a gate-based MLP which is widely adopted in recent state-of-the-art on-device LLMs [20]–[22]. The gate-based MLP consists of two input projection layers ($W_{\text{gate}} \in \mathbb{R}^{k\times d}$ and $W_{\text{up}} \in \mathbb{R}^{k\times d}$), a non-linear activation function σ, and an output projection layer ($W_{\text{down}}^{\text{T}} \in \mathbb{R}^{d\times k}$), where d and k denote the model dimension (embedding size) and hidden dimension (feed-forward size), respectively. Specifically, given an input vector $X \in \mathbb{R}^{d\times 1}$, the output of the gate-based MLP can be obtained through the following sequence of operations:

$$h_1 = \sigma(W_{\text{gate}} \cdot X), \tag{1}$$

$$h_2 = W_{\text{up}} \cdot X, \tag{2}$$

and

$$\text{MLP}(X) = W_{\text{down}}^{\text{T}} \cdot (h_1 \circ h_2) \tag{3}$$

with $\circ$ denoting element-wise multiplication.

Activation Sparsity: By inducing sparsity in the input X, the number of effective multiplications in the three GEMV operations of Eqs. (1-3) can be reduced, since the contributions from zero entries can be safely skipped. Moreover, when h_1 is computed first, any element of h_2 corresponding to a zero entry in h_1 will be multiplied by zero during the element-wise product $h_1 \circ h_2$. Consequently, the associated memory loading involving W_{up} for those positions can also be removed.

While some methods [9], [17] enlarge the number of zeros in X to exploit *input* activation sparsity, this strategy sees limited adoption because memory loading for W cannot be eliminated without column-major weight restructuring. Consequently, most recent work focuses on *output* activation sparsity. That is, in Eq. (1), many elements of h_1 become

zero after the activation function σ, and such sparsity can be leveraged to skip unnecessary multiplications and memory loading. To further maximize output activation sparsity, Mirzadeh et al. [11] proposed replacing smooth activation functions such as SiLU [23] or GELU [24], which tend to produce many non-zero values, with ReLU to increase output activation sparsity. With subsequent fine-tuning, this *ReLU-fication* strategy has been shown to preserve accuracy with negligible degradation, and has inspired a wide range of output sparsity-based techniques. For example, DejaVu [12] and PowerInfer [25] predict the sparsity pattern by introducing two additional fully-connected (FC) layers, though they have the drawback of requiring additional training.

Training-free Sparsity Exploitation: To avoid the overhead of additional training, training-free approaches have been recently proposed for both ReLU-fied LLMs [10], [18] and SiLU-based LLMs [14] to exploit output activation sparsity.

SparseInfer [10] *predicts* output activation sparsity. When computing $h_{1,i}$ in Eq. (1), instead of performing the exact inner product between $W_{\text{gate},i}$ and X, it examines the partial products at the bit level. By leveraging XOR operations, it determines only the sign of each partial product without computing their exact magnitudes. If the number of negative partial products (N_{neg}) significantly exceeds the number of positive ones (N_{pos}), the final inner product is highly likely to be negative, and thus the activation after ReLU will be zero. In such cases, SparseInfer directly predicts output activation sparsity and skips the corresponding multiplications and memory loading, thereby reducing both computational and memory overhead. To mitigate inevitable inaccuracies from this approximation, a tunable margin parameter $\alpha > 0$ is introduced, and sparsity is declared if

$$\alpha \cdot N_{\text{pos}} - N_{\text{neg}} < 0. \tag{4}$$

Increasing α yields a more conservative prediction, i.e., less sparsity.

Grasp [18] extends the above sign-based predictor by incorporating magnitude information and explicitly handling outliers. Instead of relying solely on the signs, it partitions their entries into four bins, i.e., *large positive*, *small positive*, *large negative*, and *small negative*, using a magnitude threshold, and bases its decision on the resulting *weighted* counts. Examining the pairwise products of these four bins reveals three natural categories based on absolute value: *large* × *large*, *small* × *small*, and an intermediate group containing the remaining combinations. For each of these three categories, the method computes $N_{\text{pos}} - N_{\text{neg}}$ separately and calculates the final prediction based on their weighted sum as follows:

$$N_{\text{total}} = \sum_{i=1}^{3} S_i \cdot (N_{\text{pos},i} - N_{\text{neg},i}) < 0 \tag{5}$$

where S_i denotes the scaling factors for each group, which in Grasp are chosen as either average values or predetermined constants. N_{total} serves as a proxy score for the dot product and the activation sparsity is predicted to exist only when N_{total} is negative.

Unlike SparseInfer or Grasp, CATS [14] employs a magnitude-based *thresholding* strategy to increase the degree of sparsity in SiLU. Instead of directly using h_1, it applies a thresholding function $\tau(\cdot)$ to induce sparsity:

$$\tau(h_{1,j}) = h_{1,j} \text{ if } |h_{1,j}| \geq th, \text{ otherwise } 0, \tag{6}$$

where th is a threshold determined from calibration runs. Note that, unlike SparseInfer or Grasp, h_1 must still be fully computed, and sparsity is exploited only in h_2 according to the thresholding result. A further limitation of this approach is that once the sparsity level grows beyond a certain point, accuracy degrades significantly, which requires fine-tuning to compensate for the loss.

Sparse Execution Optimizations: Orthogonal to activation-sparsity prediction, another line of work focuses on improving the execution efficiency of sparse computations through compressed sparse representations and sparse-matrix execution techniques [26]–[28]. These approaches improve sparse computation efficiency by reducing memory traffic and optimizing sparse data representations and execution, rather than by controlling how sparsity is generated. In on-device LLM inference, where decoding is typically performed with a batch size of one, the benefits of compressed sparse representations may be reduced by additional indexing and metadata overhead as well as irregular memory accesses, depending on the sparsity pattern and hardware architecture. Accordingly, the present work focuses on runtime adaptation of sparsity-related parameters while using the original sparse execution backend. These techniques are complementary to ACAS and can be naturally integrated in future work.

Runtime Adaptation in DNN/LLM Inference: Prior work has explored runtime adaptation mechanisms for deep neural network inference to meet quality-of-service (QoS) objectives such as latency, accuracy, and resource constraints. Systems such as DeepRT [29] and DMS [30] employ control-theoretic or adaptive strategies to dynamically adjust inference configurations under varying runtime conditions. More recently, AdaScale [31] introduces automated adaptation loops to dynamically scale model execution on mobile platforms. Related efforts have also explored runtime-aware adaptation of DNN execution under resource or performance constraints [32]. While these approaches demonstrate the effectiveness of feedback-driven adaptation, they primarily operate at the model or system level (e.g., adjusting model size, execution frequency, or resource allocation), rather than directly controlling fine-grained computational behaviors within neural network layers.

In the context of sparse LLM inference, prior work such as SLIM [33] explores runtime configurability of sparsity via threshold-based mechanisms. However, these approaches are typically designed with specific hardware architectures or execution pipelines in mind, and rely on predefined heuristics or static calibration, rather than incorporating closed-loop feedback to continuously refine sparsity decisions during inference. Moreover, this method is closely coupled to its underlying sparsity mechanisms, limiting its applicability across different activation-sparsity techniques.

More broadly, ACAS abstracts the sparsity-control problem across different activation-sparsity techniques into a common feedback-driven optimization framework. In contrast to prior

work, ACAS introduces a feedback-driven control loop that directly operates on activation sparsity hyperparameters at inference time. Unlike prior systems that adapt high-level configurations, ACAS leverages fine-grained feedback signals derived from sparsity prediction quality (e.g., confusion matrix statistics and output deviations) to continuously adjust sparsity behavior at the layer level.

III. Proposed Method: *ACAS*

We hypothesize that activation-sparsity parameters exhibit local optimality that varies with run-time conditions. Leveraging this insight, we adopt a PID controller [34], a widely used method for continuously tracking and correcting error signals, to adapt these parameters during inference.

A. Overview

During on-device LLM inference it is common to assume a CPU–GPU heterogeneous architecture. Prior efforts have attempted to exploit the otherwise idle CPU resources during decoding [25], [35], [36], but splitting the core GEMV computations across devices typically incurs prohibitive communication overhead and can also raise power/thermal concerns by heavily loading the CPU. Instead, as illustrated in Fig. 1, we propose to leverage the CPU to drive a PID-controller–based optimization of activation sparsity, allowing the GPU to execute the main GEMV workload while the CPU adaptively tunes sparsity-related parameters with minimal overhead.

As shown in Fig. 1, we designed the framework with the following key considerations. First, the parameter-optimization process must not stall GPU execution so as to avoid any interference with ongoing decoding. Second, it must receive real-time feedback on the quality of current activation-sparsity decisions so that parameter updates can promptly reflect runtime conditions. Finally, the framework is method-independent, allowing ACAS to be integrated into different activation-sparsity techniques without requiring major architectural changes.

To realize these goals, we collect feedback only *periodically* (e.g., every N-th token) rather than always. As illustrated in Fig. 1, the lightweight sparse GEMV (highlighted by the green circular arrow) runs for every token. In addition, at periodic intervals a dense GEMV (without skipping) is executed to *measure* the accuracy of the current activation-sparsity decision. This is done by constructing a confusion matrix. That is, if the sparse GEMV labels a unit as skip and the dense GEMV confirms that, the case is counted as a true positive (TP). For the same prediction, if it turns out that it is not actually skippable, it is a false positive (FP). Conversely, if the sparse GEMV decides not to skip and the dense GEMV confirms that it is indeed not skippable, it is a true negative (TN). If this prediction turns out to be wrong, this case is counted as a false negative (FN). Additionally, the normalized L2 error between the dense and sparse GEMV outputs can also serve as an accuracy metric.

This metric is transmitted to the host, where it is appropriately transformed into an error signal and then passed to the PID controller. Based on this error, the PID controller computes updated parameter values and sends them back to the GPU for application.

B. Interface and Implementation

We implement ACAS in the CUDA backend of the open-source LLM engine llama.cpp [37]. ACAS runs two paths concurrently. The inference path (green flow) is the standard sparse forward pass. The optimization path (yellow flow) is lightweight and adjusts the sparsity parameters during generation without stalling the forward pass. Together the two paths form an asynchronous producer-consumer pipeline. The producer (GPU) compares the sparse output against the dense one and sends the result to the host. The consumer (CPU) turns that result into a parameter update for the next token.

Producer Pipeline (GPU): Periodically, each layer's MLP runs a dense GEMV alongside the sparse one and compares the two outputs. The comparison reduces to a payload of two to four 32-bit words (8 to 16 bytes). The host later turns this payload into a quality metric, which we define below. The producer copies this payload to the host with a non-blocking *cudaMemcpyAsync* on a dedicated CUDA stream. The inference stream never issues a synchronizing call, so the forward pass keeps running while the transfer completes in the background. The exchange is never a bottleneck.

Background Consumer (CPU): The consumer is a background thread that runs the update off the critical path. Instead of polling, it waits on a condition variable. When a transfer completes, a host callback registered with *cudaLaunchHostFunc* enqueues the payload and wakes the thread. The thread turns the payload into the quality metric and its error signal e, updates the per-layer parameters through the controller, and applies them on the next token. Because it only wakes when a transfer completes, the thread uses no CPU while it is idle. Its work also overlaps with the next GEMV, which forms a natural pipeline between the CPU-side optimization and the GPU-side inference.

C. Feedback Signals

Each dense check produces a quality metric $m^{(t)} \in [0, 1]$ that quantifies how accurate the current activation-sparsity decision is. The controller acts on this metric identically, so the metric is the only component a method needs to supply in order to be controlled by ACAS.

We define the error $e^{(t)}$ using a conservative function that evaluates deviations from the target metric while emphasizing performance near extreme values as follows:

$$e(t) = \text{sign}(\Delta_{\text{logit}}) \cdot \left(\frac{2}{1 + e^{-\lambda |\Delta_{\text{logit}}|}} - 1 \right), \tag{7}$$

where the logit difference Δ_{logit} is $\ln\left(\frac{m_{\text{target}}}{1-m_{\text{target}}}\right) - \ln\left(\frac{m^{(t)}}{1-m^{(t)}}\right)$. Here, $m^{(t)}$ represents the current metric value at iteration t computed from the appropriate evaluation comparing the sparse computation with the dense reference. The target m_{target} is set by the user as a real value close to the desired extreme (e.g., 0.997 for precision targets). The sensitivity parameter $\lambda > 0$ controls the response magnitude, and metric values are clamped to $[10^{-10}, 1 - 10^{-10}]$ for numerical stability.

It is important to note that m_{target} should not be set to a *perfect* value: if it is, the controller would continually favor

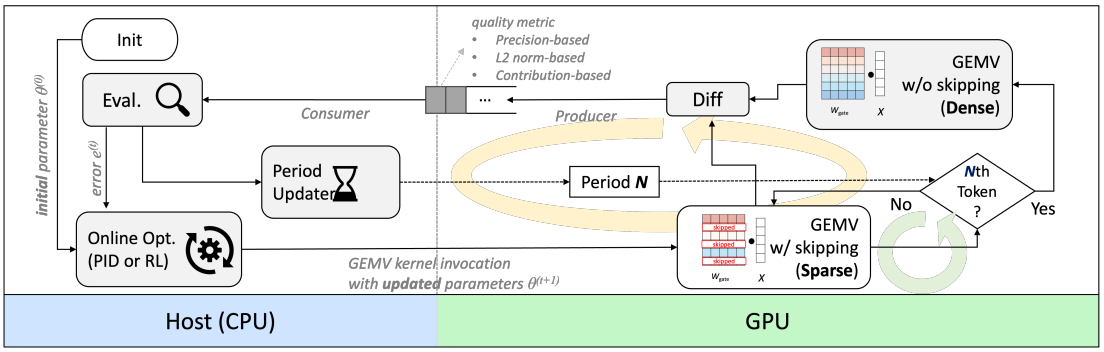

Fig. 1: Overview of the proposed ACAS framework. ACAS dynamically controls activation sparsity during LLM inference through a closed-loop feedback mechanism. Statistics collected from GPU execution (i.e., precision-based, L_2 norm-based, and contribution-based metrics) are evaluated on the host (CPU) to compute control signals via a PID or RL-based controller, which adaptively updates sparsity-inducing parameters. These updated parameters are then applied to subsequent GEMV kernels.

accuracy at the expense of speed, for example, by persistently increasing α in Eq. (4). Choosing a high but not ideal target encourages the controller to drive more aggressive skipping once the desired accuracy level is safely exceeded.

ACAS uses three quality metrics, all in $[0, 1]$ and all evaluated against a user-specified target through Eq. (7). That is, $m^{(t)}$ can be instantiated using one of the following three quality metrics:

1) the precision of the activation sparsity prediction, (TP)/(TP+FP), obtained from the confusion matrix that compares the predicted skips with the dense reference (precision-based target),
2) one minus the normalized L_2 error between the dense and sparse post-activation vectors (L_2 norm-based metric), and
3) one minus the residual-normalized error between the dense and sparse MLP output vectors (contribution-based metric). Here, the metric is defined as

$$m^{(t)} = \text{clamp}\left(1 - \frac{\|\mathbf{y}^{\text{dense}} - \mathbf{y}^{\text{sparse}}\|}{\|\mathbf{r}\|},\ 0,\ 1\right), \quad (8)$$

where $\mathbf{y}^{\text{dense}}$, $\mathbf{y}^{\text{sparse}}$, and $\mathbf{r}$ are the dense and sparse MLP outputs, and the residual hidden state vector to which the MLP output is added, respectively.

D. Online Optimization

Given the error $e^{(t)}$, ACAS performs online optimization of each tunable parameter θ. In this work, we consider two optimization methods: a classical PID controller and a reinforcement-learning (RL) agent. Since both operate solely on the error signal $e^{(t)}$, either can be paired with any of the quality metrics defined above.

1) PID Controller: The operation of the PID controller is described by the following update rule:

$$\theta^{(t+1)} = \text{clamp}\Big(\theta^{(t)} + K_p e^{(t)} + K_i \sum_{i=0}^{t} e^{(i)} + K_d\big(e^{(t)} - e^{(t-1)}\big),\ \theta_{\min}, \theta_{\max}\Big), \quad (9)$$

where $\theta^{(t)}$ is the value of the tunable parameter after iteration t and $e^{(t)}$ denotes the tracking error at iteration t. K_p, K_i, and K_d are the proportional, integral, and derivative gains that weight the instantaneous error, the cumulative error $\sum_{i=0}^{t} e^{(i)}$, and the change in error $e^{(t)} - e^{(t-1)}$, respectively. The operator $\text{clamp}(\cdot, \theta_{\min}, \theta_{\max})$ projects the updated value onto the admissible range $[\theta_{\min}, \theta_{\max}]$, preventing the parameter from exceeding the predefined bounds. In practice, the PID gains are calibrated once on a single representative benchmark and reused across all models, activation functions, and sparsity methods. As shown in the experiments, moderate perturbations (±30%) of these gains yield similar convergence and steady-state behavior, indicating that ACAS is not overly sensitive to the exact choice of gain values.

2) RL-based Optimization: As an alternative to the PID controller, we also implement a per-layer tabular Q-learning agent [38] for online optimization. Each layer maintains an action-value table $Q(s, a)$. The state s maps the tracking error $e^{(t)} = m^{(t)} - m_{\text{target}}$ into one of seven bins with edges at $\pm\sigma$, $\pm 2\sigma$, and $\pm 3\sigma$, where σ is the standard deviation of the metric over the last 40 dense checks. An action $a \in \{-8, -3, -1, 0, 1, 3, 8\}$ is a relative step size that is scaled per method into an update to θ. After each dense check the RL agent observes the reward $r^{(t)} = -|e^{(t)}|$, which is maximized when the metric sits at its target, and applies the Q-learning update

$$Q(s_{t-1}, a_{t-1}) \leftarrow Q(s_{t-1}, a_{t-1}) + \eta\, \delta^{(t)}, \quad (10)$$

where the temporal-difference error $\delta^{(t)} = r^{(t)} + \gamma \max_{a'} Q(s_t, a') - Q(s_{t-1}, a_{t-1})$.

E. Flexible Update Periods

As discussed earlier, ACAS relies on a PID controller to determine appropriate sparsity parameters at runtime, which requires periodically performing dense computations to obtain reference outputs. Although such measurements are infrequent, they can introduce unnecessary overhead when the system is already in a stable state. On the other hand, when the workload or input characteristics change rapidly, a fixed update

period may be insufficient to promptly reflect these changes, resulting in delayed adaptation and suboptimal performance.

To address this limitation, we extend ACAS with a flexible update mechanism that dynamically adjusts the period between control updates. The key idea is to increase the update interval when the system remains stable, thereby reducing overhead, and to decrease it when larger deviations from the target quality are observed, enabling faster adaptation. The resulting algorithm adaptively balances responsiveness and efficiency, as described in Algorithm 1.

Algorithm 1 Flexible Update Period

Require: Target quality m_{target}; tolerance $\epsilon = 0.01$; initial period $N_0 = 200$; bounds $N_{\min} = 50$, $N_{\max} = 500$; scale factors $\gamma_{\text{inc}} = 1.5$, $\gamma_{\text{dec}} = 0.5$

1: $N \leftarrow N_0$
2: **for** each token step $t = 1, 2, \ldots$ **do**
3: **if** $t = t_{\text{next}}$ **then**
4: Compute dense $\mathbf{y}_t^{\text{dense}}$ and sparse $\mathbf{y}_t^{\text{sparse}}$ output
5: Measure quality: $m^{(t)} \leftarrow \text{Metric}(\mathbf{y}_t^{\text{sparse}}, \mathbf{y}_t^{\text{dense}})$
6: $e_t \leftarrow m^{(t)} - m_{target}$
7: **if** $e_t \geq \epsilon$ **then**
8: $N \leftarrow \min\big(\lfloor \gamma_{\text{inc}} \cdot N \rfloor,\ N_{\max}\big)$
9: **else if** $e_t \leq -\epsilon$ **then**
10: $N \leftarrow \max\big(\lfloor \gamma_{\text{dec}} \cdot N \rfloor,\ N_{\min}\big)$
11: **end if**
12: $t_{\text{next}} \leftarrow t + N$
13: **end if**
14: **end for**

The update period is adaptively adjusted within the predefined range $[N_{\min}, N_{\max}]$, based on the deviation between the measured quality and the target. Specifically, the error term $e_t = m^{(t)} - m_{target}$ (line 6) determines how the update interval evolves. When $e_t \geq \epsilon$, the system is considered to be operating conservatively (i.e., achieving higher quality than required), and thus the update period is increased within the allowed maximum bound (line 8), reducing unnecessary control overhead. In contrast, when $e_t \leq -\epsilon$, the observed quality falls below the target, indicating that more frequent updates are needed to promptly correct the sparsity parameters; accordingly, the update period is decreased within the minimum bound (line 10). When the error remains within the tolerance range, the current update period is maintained, avoiding unnecessary oscillations.

IV. Enhanced Activation Sparsity with ACAS

This section demonstrates how ACAS can be applied to existing activation-sparsity techniques.

A. ACAS-SparseInfer and ACAS-Grasp

In SparseInfer [10], the authors set the sparsity parameter α in Eq. (4) empirically. Based on the observation that early layers are more sensitive to skipping while later layers naturally exhibit higher sparsity, the authors assign $\alpha > 1$ to the first 20 layers and fix α to 1.0 for all remaining layers. For example, in the configuration denoted as "SparseInfer $\alpha = 1.03$," the first 20 layers use $\alpha = 1.03$ while the remaining layers use $\alpha = 1.0$.

We extend SparseInfer by applying ACAS so that the α values are individually optimized for every layer (*ACAS–SparseInfer*). In this setting, the variable θ in Eq. (9) is defined as a vector of dimension L, $\big[\alpha_0, \alpha_1, \ldots, \alpha_{L-1}\big]$, which represents the per-layer α values for a model with L layers. $\theta_{\min,i}$ is 0 as α by definition is a positive value and $\theta_{\max}$ is left unbound to maximize sparsity. In ACAS-SparseInfer, the quality metric $m^{(t)}$ is defined using either the precision-based or the contribution-based metric described in Section III-C.

In the original Grasp [18] paper, there is no tunable parameter analogous to the α of SparseInfer as shown in Eq. (5). To enable the application of ACAS, we slightly extend Eq. (5) by introducing a tunable parameter β as follows:

$$N_{\text{total}} < (1 - \beta) \cdot d/2. \tag{11}$$

When $\beta < 1.0$, the skip condition becomes more permissive, whereas $\beta > 1.0$ makes the skip criterion more conservative. Because the value of N_{total} should be normalized by the vector dimension, we scale it by $d/2$.

By optimizing β for each layer, θ becomes $\big[\beta_0, \beta_1, \ldots, \beta_{L-1}\big]$. In ACAS-Grasp, we set $\theta_{\min,i} = 0$ and introduce an upper bound $\theta_{\max,i} = 1.03$. This choice is based on an empirical observation: with Grasp, increasing β beyond this range provided little additional benefit and occasionally led to overly conservative updates. The definitions of $e^{(t)}$ and $m^{(t)}$ are identical to those in ACAS-SparseInfer.

B. ACAS-CATS

In CATS [14], the threshold th in Eq. (6) is computed from calibration runs. The authors use randomly selected samples from the RefinedWeb dataset [39] to collect post-MLP activations for each layer. Using the resulting distribution, they compute the threshold that skips the kth percentile of values. For example, CATS-50% refers to the static thresholds that result in zeroing out 50% of the post-MLP activations. On average, this yields about a 50% reduction in the memory overhead of W_{up}.

We extend CATS by applying ACAS (ACAS-CATS) so that the th values are individually optimized for every layer. In this setting, the variable θ in Eq. (9) is defined as a vector of dimension L, $\big[th_0, th_1, \ldots, th_{L-1}\big]$, which represents the per-layer th values for a model with L layers. In this setup, θmin is 0 as we only care about the absolute value of the post-MLP activations for skipping. θmax is left uncapped to maximize sparsity. In ACAS-CATS, the quality metric $m^{(t)}$ is defined using either the L_2 norm-based or the contribution-based metric described in Section III-C.

C. ACAS-CATS with Gate Prediction (GP)

As discussed earlier, CATS determines activation sparsity by first fully computing h_1 in Eq. (1) and then applying thresholding for h_2. Consequently, even if ACAS-CATS efficiently discovers an optimal threshold, its benefit remains confined to h_2 (thus W_{up}), which limits the overall performance gain.

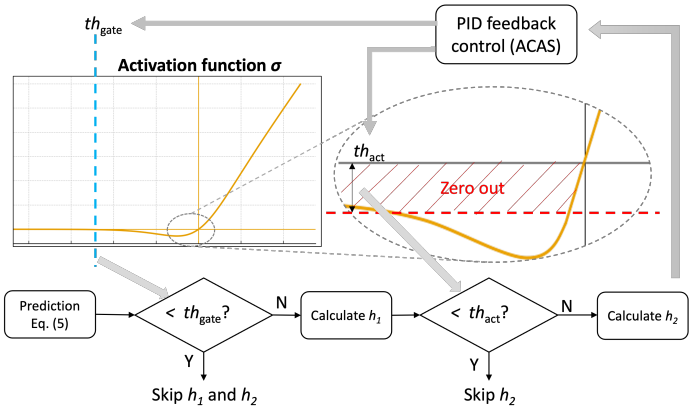

Fig. 2: Enhanced ACAS-CATS with a gate prediction.

This is a fundamental limitation of methods applied to models based on non-ReLU activations.

We propose a method to overcome this limitation through ACAS. Specifically, based on the fact that, in non-ReLU activations, inputs with sufficiently large negative magnitude produce outputs that are effectively zero, we let ACAS adaptively tune a threshold to identify such inputs and partially skip the computation of h_1. This enhanced approach, referred to as ACAS-CATS with Gate Prediction (ACAS-CATS-GP), introduces gate prediction into ACAS-CATS so that activation sparsity can be exploited at the h_1 stage as well. As illustrated in Fig. 2, the method operates in two stages:

1) Gate Prediction: a lightweight proxy for $W_{\text{gate}}X$ is computed via Eq. (5), and values below th_{gate} are skipped, avoiding computation and memory loading for both h_1 and h_2.
2) Activation Thresholding: For elements not skipped by gate prediction, we compute the full $W_{\text{gate}}X$, apply $\sigma(\cdot)$, and threshold the result using th_{act}.

Both thresholds are individually optimized per layer with variables $\theta_1 = \left[th_{\text{gate},0}, \ldots, th_{\text{gate},L-1}\right]$ for the gate predictor and $\theta_2 = \left[th_{\text{act},0}, \ldots, th_{\text{act},L-1}\right]$ for the activation threshold. ACAS-CATS-GP can optimize both in two ways. The first uses a separate quality metric for each. For θ_1, we use a variant of the precision-based metric that measures the wrong skip rate, which can be directly obtained from the confusion matrix as $(FP)/(TP + FP + FN + TN)$, with the ground truth set by which values θ_2 would ultimately zero, so it learns to flag activations that will eventually be discarded. θ_2 uses the L_2 norm-based metric, as in ACAS-CATS. Alternatively, the contribution-based metric, as described in Section III-C, can be used as a single quality metric to optimize both θ_1 and θ_2.

The proposed ACAS-CATS and ACAS-CATS-GP mechanisms are conceptually related to ReLU-fication, which increases output activation sparsity by replacing smooth activation functions such as SiLU or GELU with ReLU. However, unlike ReLU-fication, which modifies the activation function and typically requires fine-tuning to recover accuracy, ACAS-CATS retains the original SiLU activation and induces sparsity through runtime-adaptive thresholding. From this perspective, ACAS-CATS can be viewed as an online and deployment-time counterpart to ReLU-fication: instead of fixing the degree of sparsification through offline model modification, ACAS continuously adjusts the sparsification level according to the observed runtime quality signal. ACAS-CATS-GP further extends this idea by identifying large negative gate inputs whose SiLU outputs are effectively negligible, thereby allowing part of the gate computation itself to be skipped. This suggests that ReLU-fication-style sparsity and runtime sparsity control are complementary and could be jointly optimized in future work.

V. Experiments

A. Experimental Setup

Experiments were conducted on a Jetson Orin 64GB with Ubuntu 22.04, JetPack 6.2, and CUDA 12.6. All ACAS implementations from Section IV are built on top of the CUDA backend of llama.cpp [37]. For the PID controller, the PID gains were set according to the scale of the error metric: when the error was derived from the confusion-matrix, they were set to $K_p = 1.0$, $K_i = 0.2$, and $K_d = 0.15$; when the L2 error was used, the gains were set to $K_p = 0.5$, $K_i = 0.1$, and $K_d = 0.05$. For the RL agent, the learning rate and discount factor were set to $\eta = 0.3$ and $\gamma = 0.85$, respectively. An ϵ-greedy policy was adopted, with ϵ decaying from 0.30 to 0.02 over the first 200 updates. For the static-period configuration, ACAS was set to update the tunable parameters once every 200 tokens. For the flexible-period configuration, a default period of 200 tokens was used, and the update schedule was dynamically adjusted following Algorithm 1. End-to-end inference speedup is evaluated based on decoding latency, defined as the average time per output token (TPOT), i.e., the inter-token generation interval, a commonly used metric for LLM inference efficiency [40]. We conducted experiments on two on-device LLM models, ProSparse-LLaMA-2 [13] (ReLU-fied LLaMA-2 [21])[1] and LLaMA-3 [22] (SiLU-based). The quality of inference is evaluated using lm-evaluation-harness [41] on three benchmarks: GSM8K [42] (8-shot), Big-Bench Hard [43] (3-shot), and MBPP [44] (3-shot) using pass@1 scoring.

B. Effectiveness/Overhead of Inference-Time Control

1) Performance and Runtime Overhead: We applied ACAS to the two activation sparsity techniques for ReLU-fied LLMs using target values close to the ideal quality level (typically around 0.98–0.997), chosen to maximize inference quality while still enabling meaningful runtime adaptation. A comprehensive evaluation over different target values is presented later in Section V-E. The experimental results are summarized in Table I. As shown in the second and third columns, we evaluated multiple combinations of quality metrics and optimization methods. For each configuration, the accuracy on GSM8K, BBH, and MBPP is reported together with the average accuracy. The last three columns report the average token generation latency (TPOT), average power consumption, and energy per generated token, respectively.

[1]As of now, there are no public LLaMA-3–based ReLU-fied models; therefore, ReLU-fied techniques are evaluated only on a LLaMA-2 variant.

TABLE I: ProSparse-LLaMA-2-7B (ReLU-fied) Results.

* Flexible update period (Algorithm 1) applied

	Signal	Ctrl	GSM8K	BBH	MBPP	Average	TPOT [ms]	Power [W]	Energy/Tok [mJ]
No Sparsity	–	–	13.04	36.11	28.80	25.98	84.14 (1.00×)	44.88	3778.6
SparseInfer	–	–	13.19	35.95	27.20	25.45	54.53 (1.54×)	39.86	2163.8
ACAS-SI	Prec.	PID	**14.10**	**36.31**	27.80	26.07	54.21 (1.55×)	39.58	2145.6
		PID*	13.95	35.35	**29.20**	**26.17**	53.65 (1.57×)	39.88	2141.2
	Contrib.	PID	13.72	35.86	28.00	25.86	**51.88** (1.62×)	40.67	2152.6
		PID*	14.03	35.71	28.20	25.98	52.29 (1.61×)	39.63	2070.6
		RL	13.04	36.29	28.60	25.98	52.19 (1.61×)	**39.56**	**2062.8**
		RL*	12.13	36.05	27.80	25.33	53.05 (1.59×)	39.65	2107.1
Grasp	–	–	13.12	34.93	27.20	25.08	49.63 (1.70×)	38.91	1931.0
ACAS-GR	Prec.	PID	13.57	35.89	27.20	**25.55**	54.02 (1.56×)	39.64	2141.4
		PID*	**13.72**	35.18	**27.60**	25.50	52.91 (1.59×)	39.58	2091.4
	Contrib.	PID	13.27	34.93	26.40	24.87	47.23 (1.78×)	38.79	1830.0
		PID*	11.68	35.18	26.60	24.49	47.52 (1.77×)	**38.64**	1837.8
		RL	12.28	36.18	27.00	25.15	**47.11** (1.79×)	38.75	**1826.2**
		RL*	10.01	**36.26**	25.20	23.82	48.05 (1.75×)	38.69	1860.2

Compared with their respective baselines, both ACAS-SparseInfer (ACAS-SI) and ACAS-Grasp (ACAS-GR) consistently improved inference quality while maintaining comparable or even lower average decoding latency. More importantly, across all evaluated configurations, ACAS achieved higher accuracy than not only the corresponding sparsity baselines but also the no-sparsity baseline, demonstrating that runtime optimization of activation sparsity can go beyond merely recovering the accuracy loss introduced by sparsity. These results validate the effectiveness of online activation-sparsity optimization for improving inference quality while preserving efficiency.

Among the evaluated quality metrics, the precision-based metric (*Prec.*) generally provided the largest improvements in accuracy, whereas the contribution-based metric (*Contrib.*) achieved a more favorable latency–accuracy trade-off by further reducing TPOT with only a modest impact on accuracy.

We next applied the same experiment to CATS and confirmed that it successfully improved accuracy. As shown in Table II, whereas the original CATS experienced an 8–9 percentage points drop in accuracy compared with the non-sparse baseline, ACAS-CATS limited the degradation to roughly 0–3 percentage points while achieving a comparable inference speed. This is particularly noteworthy because such accuracy is attained solely through online optimization, without any fine-tuning. Furthermore, ACAS-CATS-GP provides additional speedup while consistently improving accuracy over the original CATS-50% configuration. Similar to the trend observed for ReLU-fied models, the contribution-based metric (*Contrib.*) consistently tends to yield higher accuracy than the L_2-based metric while preserving its speed advantage, demonstrating that the proposed contribution metric provides a more favorable accuracy–latency trade-off across both activation-sparsity frameworks.

To sum up, these consistent improvements across SparseInfer, Grasp, and CATS suggest that ACAS is not tied to a particular activation-sparsity mechanism. Instead, ACAS provides a unified runtime optimization framework that can be readily integrated with diverse activation-sparsity techniques.

2) Per-Token Latency: As shown in the TPOT column of Table I, the additional runtime overhead introduced by ACAS is effectively amortized over the default update interval of 200 tokens. Nevertheless, because update operations are performed periodically, the latency of update-event tokens is higher than that of regular decoding tokens. To quantify this effect, we measured the latency of individual tokens and separately report the average latencies of regular and update-event tokens, as well as the worst-case latency, in Table III.

As shown in Table III, update events increase the latency of individual decoding steps. For the precision-based metric (*Prec.*), the average update latency is approximately 30% higher than that of regular decoding tokens, while the worst-case latency increases by about 50%. For the contribution-based metric (*Contrib.*), the additional dense computation results in a larger increase, with average update latency approaching 2× and worst-case latency reaching approximately 2.4× that of regular decoding tokens. Although these infrequent update events are effectively amortized in the average TPOT, applications with strict per-token latency requirements should reserve sufficient latency headroom to accommodate the worst-case update latency.

We similarly measured the per-token decoding latency for the SiLU-based models, ACAS-CATS and ACAS-CATS-GP, and summarize the results in Table IV. For ACAS-CATS, the L_2 norm-based signal incurs only a modest latency overhead, with update-event tokens taking approximately 7% longer than regular decoding tokens on average and increasing the worst-case latency by about 11%. As observed for the ReLU-fied models, the contribution-based metric (*Contrib.*) introduces a larger overhead, increasing the average and worst-case update latency by approximately 57% and 59%, respectively. ACAS-CATS-GP exhibits a somewhat higher update overhead because two prediction signals are evaluated during each update event. Nevertheless, across both quality metrics and all evaluated configurations, the additional latency introduced by update events remains bounded, with the worst-case increase

TABLE II: LLaMA 3.1-8B (SiLU-based) Results.

* Flexible update period (Algorithm 1) applied

	Signal	Ctrl	GSM8K	BBH	MBPP	Average	TPOT [ms]	Power [W]	Energy/Tok [mJ]
No Sparsity	–	–	79.38	71.05	58.20	69.54	92.27 (1.00×)	45.18	4170.2
CATS-50%	–	–	69.45	62.89	50.20	60.85	83.79 (1.10×)	44.54	3724.4
ACAS-CATS	L_2	PID	76.19	68.08	58.20	67.49	85.09 (1.08×)	44.64	3795.8
		PID*	74.98	**69.39**	56.80	67.06	85.22 (1.08×)	**43.94**	3737.1
	Contrib.	PID	77.41	68.16	**58.40**	67.99	**83.83** (1.10×)	44.47	**3727.5**
		PID*	75.13	68.27	56.40	66.60	85.55 (1.08×)	44.30	3788.9
		RL	**77.86**	68.99	58.20	**68.35**	84.60 (1.09×)	44.55	3768.0
		RL*	76.80	68.76	58.20	67.92	85.87 (1.07×)	44.13	3791.6
ACAS-CATS-GP	L_2	PID	72.10	62.29	52.80	62.40	83.37 (1.11×)	43.86	3656.1
		PID*	72.40	62.13	51.00	61.84	82.62 (1.12×)	**43.53**	3592.6
	Contrib.	PID	74.83	**68.25**	54.60	65.89	**80.95** (1.14×)	43.80	**3543.9**
		PID*	72.10	65.52	50.40	62.67	81.62 (1.13×)	43.74	3567.3
		RL	**76.27**	66.72	**56.20**	**66.40**	82.11 (1.12×)	43.84	3601.6
		RL*	72.33	65.24	51.40	62.99	82.83 (1.11×)	43.73	3623.8

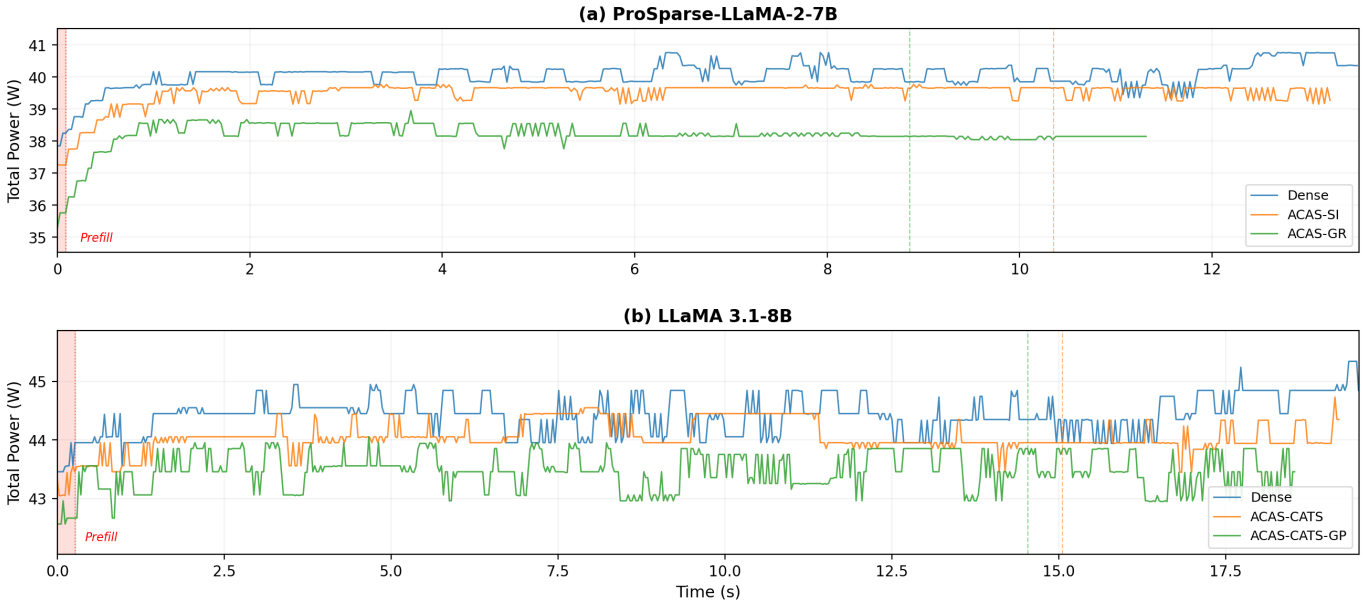

Fig. 3: Instantaneous power traces during inference (256 tokens, Jetson AGX Orin). Dashed lines indicate quality check and update events.

TABLE III: ProSparse-LLaMA-2-7B (ReLU-fied) per-token decode latency (ms).

Method	Signal	Normal	Avg. Update	Worst-case
ACAS-SI	Prec.	54.09	75.03 (+20.94)	82.70
	Contrib.	51.54	109.78 (+58.24)	118.08
ACAS-GR	Prec.	53.92	71.52 (+17.60)	78.06
	Contrib.	46.89	105.40 (+58.51)	111.77

TABLE IV: LLaMA 3.1-8B (SiLU-based) per-token decode latency (ms).

Method	Signal	Normal	Avg. Update	Worst-case
ACAS-CATS	L_2	85.05	91.62 (+6.57)	94.12
	Contrib.	83.55	131.52 (+47.97)	133.89
ACAS-CATS-GP	L_2	83.23	107.66 (+24.43)	115.52
	Contrib.	80.55	153.10 (+72.55)	155.58

limited to approximately 75 ms.

3) Power/Energy Evaluation: In terms of power and energy, ACAS maintains power efficiency despite the additional runtime adaptation. As shown in Table I and Table II, the average power consumption remains comparable to, and often lower than, that of the corresponding baselines, and in several configurations is even slightly reduced. More importantly, the energy consumed per generated token is consistently maintained or further reduced across nearly all evaluated configurations, demonstrating that the improved inference quality achieved by ACAS does not come at the cost of higher energy consumption.

We further analyze the power behavior during runtime adaptation to assess potential system-level disturbances in-

troduced by update events. As shown in Fig. 3, although ACAS performs additional dense computation at update steps, no noticeable power spikes or transient peaks are observed. The power consumption remains stable and closely follows the baseline sparse execution, with only minor fluctuations well within the normal variation range. This indicates that the additional computation is effectively amortized not only in time but also in power, and does not introduce harmful jitter or peak power overhead during inference.

4) Flexible Update Periods: We further evaluated the effect of the flexible update period proposed in Algorithm 1 by selectively enabling it for each ACAS configuration. In Table I and Table II, configurations using the flexible update period are denoted by an asterisk (*). For the precision-based quality metric, the flexible update period consistently reduced the average decoding latency (TPOT), indicating that the controller was able to safely increase the update interval once a stable operating point had been reached. In contrast, for the remaining quality metrics, the flexible update period had only a minor impact on the average latency, suggesting that the default fixed update interval was already close to an effective operating point for these signals.

For the precision-based metric, the measured value often remains close to the high target once the controller reaches a stable sparsity regime. In this case, the flexible-period rule tends to increase the update interval, thereby reducing the number of dense checks and improving TPOT. In contrast, the L_2- and contribution-based metrics provide more continuous and more sensitive measurements of output deviation. Even when the final task accuracy is maintained, these metrics can exhibit small token-to-token fluctuations around the target, causing the flexible-period mechanism to shorten the update interval more frequently. As a result, the number of update events may slightly increase, leading to a small TPOT overhead for these metrics.

Finally, we evaluated the sensitivity of the proposed flexible update period to the choice of the update-period bounds. Specifically, we repeated the experiments using alternative values of $N_{\min}$ and $N_{\max}$ instead of the default range proposed in Algorithm 1. As summarized in Table V, changing the update-period bounds resulted in only minor variations in inference accuracy. The average decoding latency (TPOT) also remained nearly unchanged across all evaluated configurations, indicating that the update-period bounds have little impact on runtime efficiency. These results indicate that the proposed flexible update mechanism is not particularly sensitive to the exact choice of the update-period range, suggesting that the default configuration provides a robust operating point.

TABLE V: Sensitivity Analysis of the flexible update period bounds (ProSparse-LLaMA-2-7B, ACAS-SI).

	$N_{\min}/N_{\max}$	GSM8K	BBH	MBPP	Avg.	TPOT [ms]
Default	50/500	14.03	35.71	28.20	25.98	52.29
Vary $N_{\min}$	25/500	14.48	35.56	28.80	26.28	53.34
	100/500	14.78	35.99	28.60	26.46	52.21
Vary $N_{\max}$	50/250	13.72	35.57	29.00	26.10	52.80
	50/1000	14.25	35.69	28.40	26.12	52.95
	50/2000	14.86	35.49	29.20	26.52	52.84

C. Runtime Adaptation Strategies

1) Comparison against Fixed-Step Heuristics: To highlight the benefit of controller-based adaptation, we compare ACAS against a fixed-step heuristic that updates the sparsity parameter α in SparseInfer by a constant increment δ at each step. We consider two representative settings: a large step ($\delta = .25$), which reacts aggressively, and a small step ($\delta = .01$), which adjusts conservatively.

The large-step heuristic achieves an average accuracy of 23.00% across GSM8K, BBH, and MBPP, while the small-step heuristic reaches 25.22%, both falling short of ACAS-SI (26.07%). The performance gap stems from the inherent limitations of fixed-step updates. The large step size leads to oscillatory behavior around the target, frequently overshooting and degrading output quality. In contrast, the small step size is too conservative to track rapidly changing conditions within the limited token budget, resulting in slow convergence. In contrast, PID control dynamically adjusts the update magnitude based on both the instantaneous error and its temporal evolution.

2) Analysis of PID Stability: Fig. 5 shows the optimization trajectories of four representative layers (0, 10, 20, and 31) when ACAS is applied to ProSparse-LLaMA-2-7B, running GSM8K. Under the baseline gains, the proposed PID controller consistently drives each layer's parameter toward its optimal value, entering the ±0.5% band of the final α within roughly 500 update steps. We observe similar convergence behavior even when the initial α is set to a smaller (*init-low*, $\alpha = 0.98$) or larger (*init-high*, $\alpha = 1.08$) value. This behavior indicates that ACAS rapidly corrects poor initializations and exhibits stable convergence across layers. To further assess robustness, we perturbed all PID gains by ±30% (*gain+30* and *gain-30*) around the baseline values. Despite these changes, all layers converged to the same steady-state α and maintained similar settling times and steady-state error, demonstrating that ACAS is not overly sensitive to the exact gain values and remains stable under moderate perturbations.

3) Alternative Controllers: To demonstrate that ACAS is not tied to a specific optimization algorithm, we replaced the PID controller with a lightweight tabular Q-learning agent while keeping the remaining framework unchanged. The corresponding results are reported in the RL/RL* rows of Table I and Table II. Overall, the RL-based controller achieves accuracy–latency trade-offs comparable to those of the PID controller across both ReLU-fied and SiLU-based models. For example, under the contribution-based metric, RL achieves an average accuracy of 25.98% for ACAS-SI compared with 25.86% using PID, while maintaining a similar TPOT (52.19 ms vs. 51.88 ms). Similar trends are observed for ACAS-CATS and ACAS-CATS-GP, where RL consistently attains accuracy comparable to or slightly higher than that of PID while preserving similar inference speed.

These results suggest that the effectiveness of ACAS primarily stems from its runtime optimization framework rather than from a particular controller implementation. In other words,

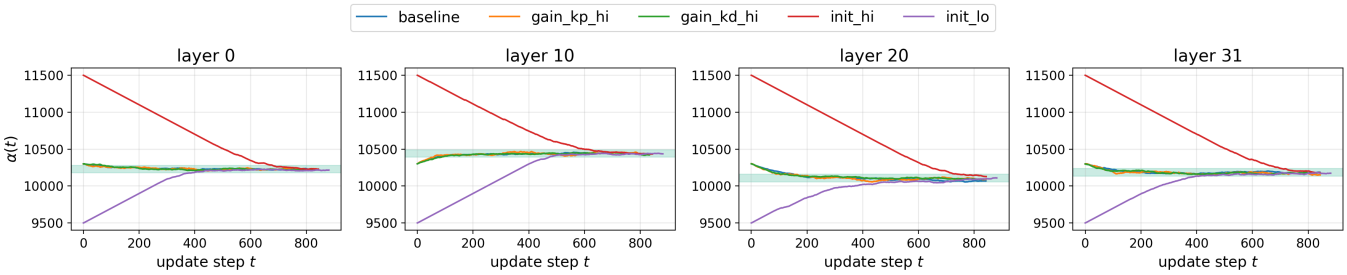

Fig. 4: Per-layer controller $\alpha(t)$ for layers 0, 10, 20, and 31.

the choice of optimization algorithm has a relatively small impact compared with the choice of the quality metric used for feedback.

It is worth noting that the RL implementation in this work intentionally uses the same interface and feedback signals as the PID controller so that only the optimization algorithm is changed. This design isolates the effect of the controller itself and serves to validate the controller-agnostic nature of ACAS. Future work may further investigate RL-specific state representations, richer observations, and alternative reward formulations that more fully exploit the potential of learning-based optimization.

D. *Analysis of Runtime Adaptation*

1) Context-Dependent Sparsity: To investigate whether online adaptation is truly necessary instead of static optimization, we examine whether the optimal sparsity configuration remains stable or changes as the inference context evolves. To this end, we use ACAS-SparseInfer and construct a workload by repeatedly executing three BBH subtasks—*dyck_languages*, *formal_fallacies*, and *causal_judgement*—in sequence. Fig. 5 visualizes the evolution of the optimized layer-wise sparsity parameters over multiple cycles. Although the same subtasks are revisited repeatedly, ACAS consistently converges to distinct sparsity patterns depending on which subtask is being executed. For example, layers 16–20 adopt relatively aggressive sparsity during *dyck_languages*, whereas the controller increases the corresponding α values after the workload transitions to *formal_fallacies*, resulting in a more conservative skipping policy. When the workload subsequently returns to *dyck_languages*, ACAS again adapts toward the previous, more aggressive sparsity configuration.

These observations suggest that the optimal activation-sparsity configuration is determined not only by the underlying model or sparsity-exploitation technique but also by the inference context itself. Consequently, a single statically calibrated configuration cannot remain optimal across changing workloads. Instead, runtime adaptation, as provided by ACAS, is required to continuously track context-dependent optimal operating points and maintain an effective accuracy–efficiency trade-off.

2) Analysis of Optimized Sparsity: To further analyze the sparsity parameters optimized by ACAS, Fig. 6 summarizes the layer-wise sparsity obtained for all benchmark–method combinations on the ReLU-fied model. The top row shows the oracle sparsity that would be obtained if activation sparsity were predicted with perfect accuracy, providing an upper bound on the exploitable sparsity for each benchmark.

Overall, ACAS successfully adapts its sparsity configuration to the characteristics of each benchmark for both SparseInfer and Grasp. For example, the oracle sparsity of BBH becomes noticeably higher than that of GSM8K and MBPP in the intermediate and later layers. ACAS automatically captures this trend, increasing the sparsity of both ACAS-SI and ACAS-GR on BBH while maintaining more conservative configurations for the other benchmarks. These observations indicate that the optimal sparsity configuration depends on the input workload rather than being fixed for a given model.

A particularly interesting observation can be made in the penultimate (second-to-last) layer of ACAS-SI. To better understand why ACAS-SI consistently selects a more conservative sparsity configuration for this layer, we further analyze the outliers[2] on the GSM8K benchmark. We found that the penultimate layer contains substantially more severe outliers than the remaining layers. Specifically, the penultimate layer exhibits substantially more severe and more frequent outliers than the remaining layers, with a maximum outlier of 61.6σ (vs. 14.3σ on average across layers) and 9.2 outliers per input vector (vs. 6.9 on average). Since SparseInfer does not explicitly account for such outliers, aggressive skipping in this layer can easily degrade prediction accuracy. ACAS autonomously identifies this behavior and adapts by selecting a more conservative sparsity configuration. In contrast, Grasp explicitly incorporates outlier handling into its prediction mechanism, allowing ACAS-GR to maintain a much more aggressive sparsity configuration in the same layer. This result demonstrates that ACAS automatically adapts not only to the input benchmark but also to the characteristics of the underlying sparsity-exploitation technique.

We performed a similar analysis for the SiLU-based methods, and the results are summarized in Fig. 7. Note that CATS exploits activation sparsity by applying magnitude thresholding, where activations below a predefined threshold are treated as zero. Consequently, there is no corresponding oracle sparsity that provides an upper bound on the exploitable sparsity. Nevertheless, the same benchmark-dependent trend can be observed. In particular, BBH consistently exhibits higher layer-wise sparsity than GSM8K and MBPP, especially

[2]Following the definition used in Grasp [18], an outlier is an element in the input vector whose magnitude exceeds the mean by more than 6σ.

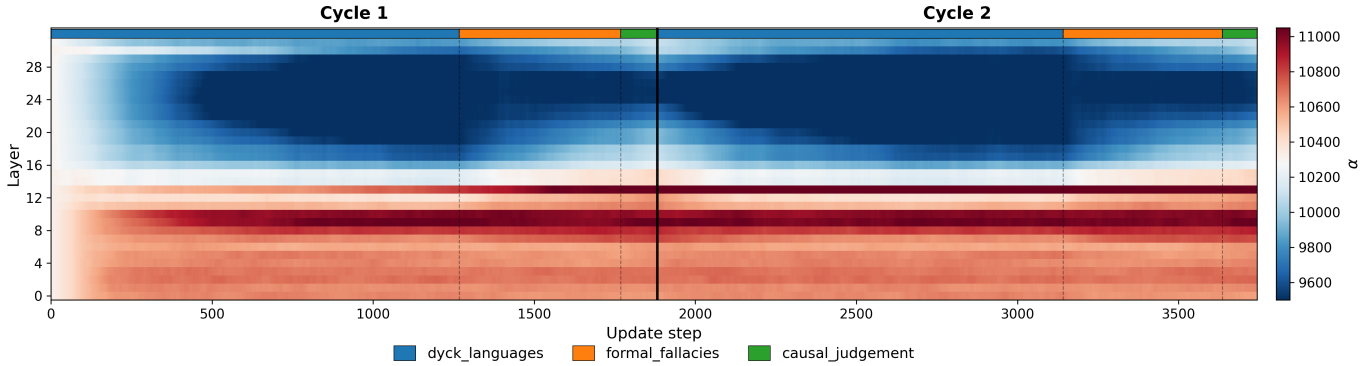

Fig. 5: Per-layer $\alpha(t)$ while cycling through selected BBH subtasks.

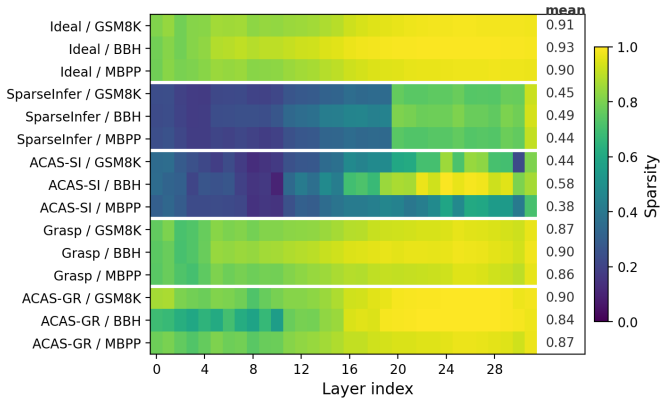

Fig. 6: Per-layer sparsity across different benchmarks on ProSparse-LLaMA-2-7B (ReLU-fied).

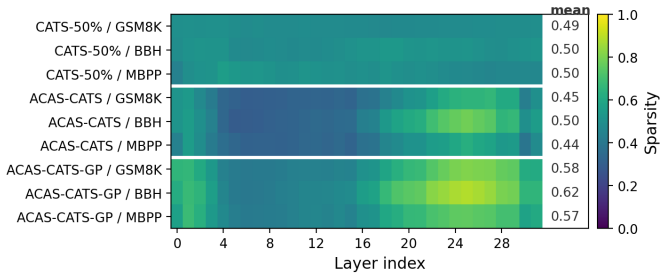

Fig. 7: Per-layer sparsity across different benchmarks on LLaMA 3.1-8B (SiLU-based).

in the later layers, and both ACAS-CATS and ACAS-CATS-GP successfully capture this characteristic by adopting more aggressive sparsity configurations for BBH. This consistency across both ReLU-fied and SiLU-based frameworks further supports the observation that the optimal sparsity configuration is strongly workload-dependent and that ACAS can automatically adapt its optimization strategy to different inference contexts.

E. Design Space Exploration (DSE) with ACAS

In the previous experiments, m_{target} was set to the highest level to emphasize accuracy improvement. Now, to demonstrate ACAS's DSE capability, we performed a multi-objective optimization of the accuracy–speed trade-off by varying m_{target} in the ACAS optimizations.

The DSE result is shown in Fig. 8. For presentation clarity, only the contribution-based results are color-coded, since the precision-based targets vary over a much narrower range (0.99–0.999). ACAS-SI and ACAS-GR produce Pareto fronts that characterize the trade-off between speed and accuracy and include the configurations of their respective baselines. In particular, ACAS-SI identifies a Pareto front in which both the "SparseInfer $\alpha = 1.02$" and "SparseInfer $\alpha = 1.03$" configurations are *dominated*, as ACAS-SI achieves superior performance in both metrics for that setting. Furthermore, compared with the precision-based metric, the contribution-based metric explores a broader design space, particularly in the region favoring higher inference speed at the expense of a reduction in accuracy.

In the case of ACAS-GR, although the Pareto front obtained using the precision-based metric does not dominate the original Grasp configuration, it nevertheless identifies multiple operating points that improve accuracy over the original Grasp configuration while incurring only a marginal latency increase. The contribution-based metric, on the other hand, consistently discovers a superior Pareto frontier that dominates both the original Grasp configuration and the precision-based solutions. Moreover, by progressively relaxing the target quality, it further extends the Pareto frontier toward higher inference speed, enabling more efficient design space exploration.

We performed a similar design space exploration for the SiLU-based methods, and the results are shown in Fig. 9. For presentation clarity, only the contribution-based targets are color-coded in the chart, whereas the L_2-based targets in Fig. 9(a) span the range of 0.60–0.99 and the wrong-skip targets in Fig. 9(b) span the range of 0.02–0.10, and are therefore shown without color coding. As shown in Fig. 9(a), both the L_2-based and contribution-based metrics enable ACAS-CATS to identify Pareto fronts that *dominate* the original CATS-50% configuration, achieving both higher inference accuracy and lower decoding latency.

As shown in Fig. 9(b), ACAS-CATS-GP performs an even richer design space exploration because it simultaneously optimizes two sparsity-related parameters. Across all evaluated target-metric combinations, ACAS-CATS-GP consistently

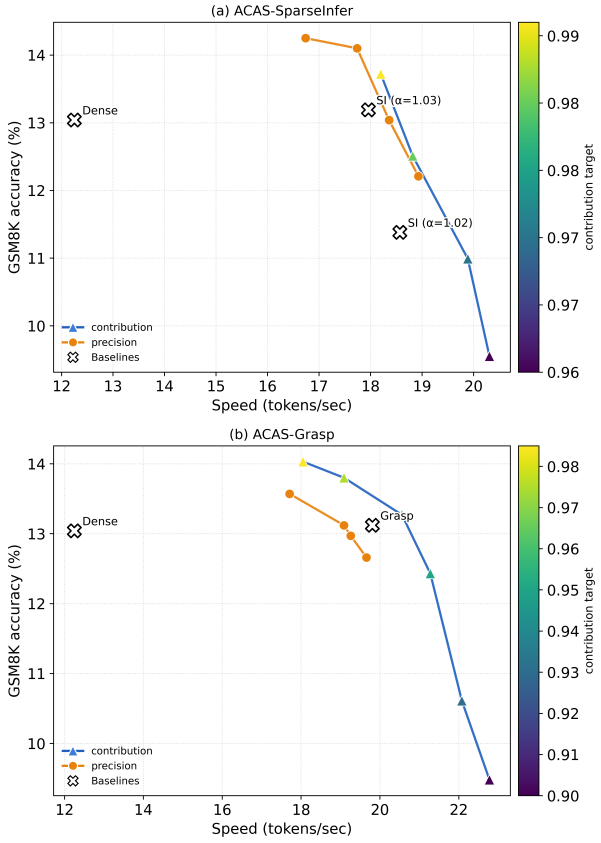

Fig. 8: Speed–Accuracy trade-off on ProSparse-LLaMA-2-7B (ReLU-fied).

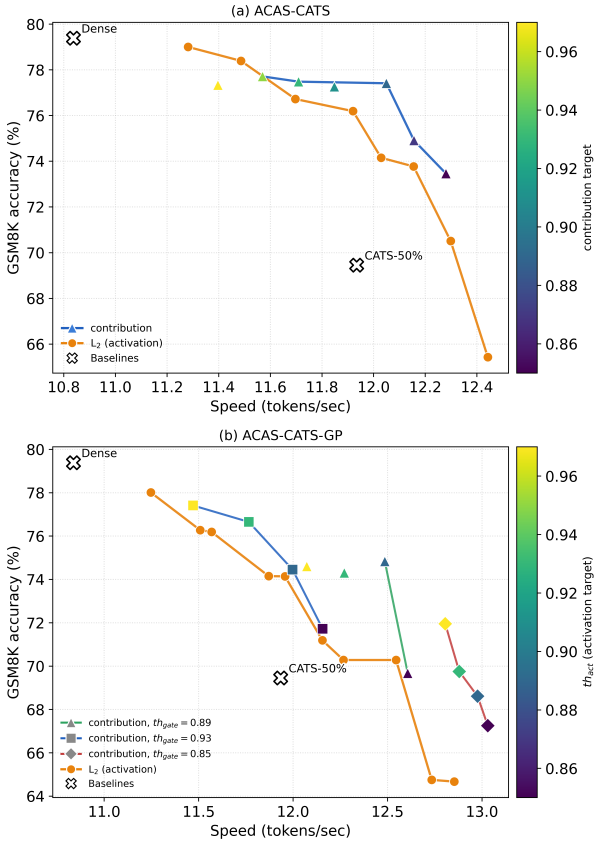

Fig. 9: Speed–Accuracy trade-off on LLaMA 3.1-8B (SiLU-based).

discovers operating points that outperform the original CATS-50% configuration in both accuracy and inference speed. These results further demonstrate that ACAS naturally extends to multi-parameter runtime optimization while effectively exploring the accuracy–latency trade-off.

VI. Conclusion

This paper presented ACAS, a lightweight inference-time framework for adaptive control of activation sparsity in on-device LLM inference. ACAS reframes the selection of sparsity-related hyperparameters as a closed-loop runtime optimization problem, allowing layer-wise sparsity configurations to be adjusted during decoding without retraining, manual calibration, or repeated offline search. By exploiting the limited host-CPU execution headroom available in CPU–GPU heterogeneous systems, ACAS performs periodic quality checks and updates sparsity parameters while the main sparse GEMV computation remains on the GPU. Across ReLU-fied and SiLU-based LLMs, and across SparseInfer, Grasp, CATS, and the proposed CATS-GP extension, ACAS consistently improves the accuracy–latency trade-off over statically configured baselines. The results also show that the framework is not tied to a particular feedback signal or controller: precision-, L2-, and contribution-based metrics, as well as PID- and RL-based optimization, can be supported through the same runtime-control interface.

Beyond improving individual sparsity methods, ACAS provides a practical mechanism for understanding and exploring activation-sparsity behavior at runtime. The optimized configurations reveal that effective sparsity levels vary across benchmarks, prompts, and layers, confirming that a single statically calibrated configuration is inherently limited. In addition, by varying the target quality metric, ACAS can be used as a lightweight design space exploration tool to trace accuracy–speed Pareto frontiers, including for SiLU-based CATS and CATS-GP configurations. Our system-level evaluation further shows that the amortized runtime, power, and energy overheads remain modest, although applications with strict per-token latency constraints should account for the bounded latency increase at update events.

Finally, while the RL-based controller evaluated in this work already demonstrates that ACAS is not tied to PID control, it intentionally reuses the same feedback interface and quality signals as the PID-based implementation. A more RL-native formulation, with richer state representations and reward functions that jointly capture quality, latency, sparsity dynamics, and workload changes, may further improve the adaptability of ACAS. Overall, ACAS demonstrates that feedback-driven runtime adaptation is a practical and general approach for making activation-sparsity techniques more portable, tunable, and robust for on-device LLM inference.

Acknowledgment

This work was supported by... .

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[2] J. Xu, Z. Li, W. Chen, Q. Wang, X. Gao, Q. Cai, and Z. Ling, "On-device language models: A comprehensive review," *arXiv preprint arXiv:2409.00088*, 2024.

[3] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale," *Advances in neural information processing systems*, vol. 35, pp. 30 318–30 332, 2022.

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D.-A. Alistarh, "Optq: Accurate post-training quantization for generative pre-trained transformers," in *11th International Conference on Learning Representations*, 2023.

[5] E. Frantar and D. Alistarh, "Sparsegpt: Massive language models can be accurately pruned in one-shot," in *International conference on machine learning*. PMLR, 2023, pp. 10 323–10 337.

[6] X. Ma, G. Fang, and X. Wang, "Llm-pruner: On the structural pruning of large language models," *Advances in neural information processing systems*, vol. 36, pp. 21 702–21 720, 2023.

[7] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[8] C.-Y. Hsieh, C.-L. Li, C.-K. Yeh, H. Nakhost, Y. Fujii, A. Ratner, R. Krishna, C.-Y. Lee, and T. Pfister, "Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes," in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 8003–8017.

[9] J. Liu, P. Ponnusamy, T. Cai, Y. Kim, B. Athiwaratkun *et al.*, "Training-free activation sparsity in large language models," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 98 302–98 322.

[10] J. Shin, H. Yang, and Y. Yi, "Sparseinfer: Training-free prediction of activation sparsity for fast llm inference," in *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, March 2025, pp. 1–7.

[11] I. Mirzadeh, K. Alizadeh-Vahid, S. Mehta, C. C. Del Mundo, O. Tuzel, G. Samei, M. Rastegari, and M. Farajtabar, "Relu strikes back: Exploiting activation sparsity in large language models," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 25 378–25 400.

[12] Z. Liu, J. Wang, T. Dao, T. Zhou, B. Yuan, Z. Song, A. Shrivastava, C. Zhang, Y. Tian, C. Re *et al.*, "Deja vu: Contextual sparsity for efficient llms at inference time," in *International Conference on Machine Learning*. PMLR, 2023, pp. 22 137–22 176.

[13] C. Song, X. Han, Z. Zhang, S. Hu, X. Shi, K. Li, C. Chen, Z. Liu, G. Li, T. Yang *et al.*, "Prosparse: Introducing and enhancing intrinsic activation sparsity within large language models," in *Proceedings of the 31st International Conference on Computational Linguistics*, 2025, pp. 2626–2644.

[14] D. Lee, J. Lee, G. Zhang, M. Tiwari, and A. Mirhoseini, "CATS: Context-aware thresholding for sparsity in large language models," in *First Conference on Language Modeling*, 2024. [Online]. Available: https://openreview.net/forum?id=v3w2a7EInO

[15] Y. Song, H. Xie, Z. Zhang, B. Wen, L. Ma, Z. Mi, and H. Chen, "Turbo sparse: Achieving llm sota performance with minimal activated parameters," *arXiv preprint arXiv:2406.05955*, 2024.

[16] Y. Luo, C. Song, X. Han, Y. Chen, C. Xiao, X. Meng, L. Deng, J. Wei, Z. Liu, and M. Sun, "Sparsing law: Towards large language models with greater activation sparsity," in *International Conference on Machine Learning*. PMLR, 2025, pp. 41 311–41 330.

[17] Z. Zhang, Z. Liu, Y. Tian, H. Khaitan, Z. Wang, and S. Li, "R-sparse: Rank-aware activation sparsity for efficient LLM inference," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=9VMW4iXfKt

[18] J. Shin, H. Yang, and Y. Yi, "Grasp: Group-based prediction of activation sparsity for fast llm inference," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–7.

[19] T. Wang, R. Fan, M. Huang, Z. Hao, K. Li, T. Cao, Y. Lu, Y. Zhang, and J. Ren, "Neuralink: Fast on-device llm inference with neuron co-activation linking," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 147–162. [Online]. Available: https://doi.org/10.1145/3676642.3736114

[20] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, "Palm: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023.

[21] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[22] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv e-prints*, pp. arXiv–2407, 2024.

[23] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," *arXiv preprint arXiv:1710.05941*, 2017.

[24] D. Hendrycks and K. Gimpel, "Gaussian error linear units (gelus)," *arXiv preprint arXiv:1606.08415*, 2016.

[25] Y. Song, Z. Mi, H. Xie, and H. Chen, "Powerinfer: Fast large language model serving with a consumer-grade gpu," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, ser. SOSP '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 590–606. [Online]. Available: https://doi.org/10.1145/3694715.3695964

[26] P. A. Lane and J. D. Booth, "Heterogeneous sparse matrix–vector multiplication via compressed sparse row format," *Parallel Computing*, vol. 115, p. 102997, 2023.

[27] A. Benatia, W. Ji, Y. Wang, and F. Shi, "Sparse matrix partitioning for optimizing spmv on cpu-gpu heterogeneous platforms," *The International Journal of High Performance Computing Applications*, vol. 34, no. 1, pp. 66–80, 2020.

[28] C.-L. Lee, C.-T. Chao, W.-H. Chu, M.-Y. Hung, and J.-K. Lee, "Accelerating ai applications with sparse matrix compression in halide," *Journal of Signal Processing Systems*, vol. 95, no. 5, pp. 609–622, 2023.

[29] W. Kang and J. Chung, "Deeprt: predictable deep learning inference for cyber-physical systems," *Real-Time Systems*, vol. 55, no. 1, pp. 106–135, 2019.

[30] W. Kang, D. Kim, and J. Park, "Dms: Dynamic model scaling for quality-aware deep learning inference in mobile and embedded devices," *IEEE Access*, vol. 7, pp. 168 048–168 059, 2019.

[31] Y. Wang, S. Liu, B. Guo, B. Zhang, K. Ma, Y. Ding, H. Luo, Y. Li, and Z. Yu, "Adascale: Dynamic context-aware dnn scaling via automated adaptation loop on mobile devices," *IEEE Internet of Things Journal*, 2025.

[32] J. Jang and H. Yang, "A runtime switchable multi-phase convolutional neural network for resource-constrained systems," *IEEE Access*, vol. 11, pp. 62 449–62 461, 2023.

[33] W. Xu, H. Choi, P.-k. Hsu, S. Yu, and T. Simunic, "Slim: A heterogeneous accelerator for edge inference of sparse large language model via adaptive thresholding," *ACM Transactions on Embedded Computing Systems*, 2025.

[34] K. J. Astrom, "Pid controllers: theory, design, and tuning," *The international society of measurement and control*, 1995.

[35] J. Fan, Y. Zhang, X. Li, and D. S. Nikolopoulos, "Parallel cpu-gpu execution for llm inference on constrained gpus," 2025. [Online]. Available: https://arxiv.org/abs/2506.03296

[36] D. Park and B. Egger, "Improving throughput-oriented llm inference with cpu computations," in *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques*, ser. PACT '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 233–245. [Online]. Available: https://doi.org/10.1145/3656019.3676949

[37] G. Gerganov, "llama.cpp," 2023, accessed: July 29, 2025. [Online]. Available: https://github.com/ggerganov/llama.cpp

[38] R. S. Sutton, A. G. Barto *et al.*, *Reinforcement learning: An introduction*. MIT press Cambridge, 1998, vol. 1, no. 1.

[39] G. Penedo, Q. Malartic, D. Hesslow, R. Cojocaru, A. Cappelli, H. Alobeidli, B. Pannier, E. Almazrouei, and J. Launay, "The refinedweb dataset for falcon llm: Outperforming curated corpora with web data, and web data only," 2023. [Online]. Available: https://arxiv.org/abs/2306.01116

[40] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626.

[41] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, "The language model evaluation harness," 07 2024. [Online]. Available: https://zenodo.org/records/12608602

[42] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano *et al.*, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[43] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. Le, E. Chi, D. Zhou *et al.*, "Challenging big-bench tasks and whether chain-of-thought can solve them," in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 13 003–13 051.

[44] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.